

Index

[illegible]

	DONE
	DONE
	DONE
	DONE
	DONE
	DONE
	DONE
	DONE
	DONE
Represent and add two polynomials using linked lists	
	DONE
	done
	done
	done
delete a node from a circular linked list at any given position	
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	done
	DONE
	DONE
	DONE
	DONE
implement stack using 2 queues and perform push pop and display on it	

done
done
done
done
done
done
deletion of a node in a doubly linked list by the value in the node
create an input restricted queue and perform enqueue dequeue and display operations
done
head of a linked list and the number k, Your task is to find the kth node from the end. If k is more than the number of nodes, then the output s
first make a loop out of the user defined linked list from the user defined point and then show it and then delete the loop
he task is to remove duplicates (nodes with duplicate values) from the given list (if it exists). Note: Try not to use extra space. The nodes ar
create a program to find the factorial of a number using recursion and iteration give choice of method to user

Program 1: done

```
//done
//linear search
#include
int main()
{
    int n;
    printf("Enter the size of the array");
    scanf("%d",&n);
    int arr[n],i,p=0;
    printf("Enter the elements of the array");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    printf("Enter the element to be searched");
    scanf("%d",&p);
    for(i=0;i {
        if(arr[i]==p)
        {
            printf("The element is found at %d",i+1);
            break;
        }
    }
}
```

Program 2: done

```
//done
//structure
#include
int main()
{
    struct student
    {
        char name[20];
        int roll;
        float marks;
    };
    struct student s1;
    printf("Enter name of the student: ");
    scanf("%s", s1.name);
    printf("Enter roll number of the student: ");
    scanf("%d", &s1.roll);
    printf("Enter marks of the student: ");
    scanf("%f", &s1.marks);
    printf("Name of the student: %s\n", s1.name);
    printf("Roll number of the student: %d\n", s1.roll);
    printf("Marks of the student: %f\n", s1.marks);
    return 0;
}
```

Program 3: del,node,without,head.c

```
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=head;
    head=new;
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
void deleteNode(struct node *del)
{
    if(del==NULL || del->n==NULL)
    {
        printf("Cannot delete the given node (it may be the last node or NULL).\n");
        return;
    }
    struct node *temp=del->n;
    del->d=temp->d;
    del->n=temp->n;
    free(temp);
}
int main()
{
    int n,i,data;
    printf("Enter the number of nodes you want to create: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ", i + 1);
        scanf("%d", &data);
        insert(data);
    }
    printf("Initial list: ");
    display(head);
    if(head!=NULL && head->n!=NULL)
```

```
{
deleteNode(head);
}
printf("List after deletion: ");
display(head);
return 0;
}
```

Program 4: test2.c

```
#include
#include
struct node {
    int coef;
    int exp;
    struct node *next;
};
struct node *head1 = NULL;
struct node *head2 = NULL;
struct node *head3 = NULL;
void insert1(int coef, int exp) {
    struct node *new = (struct node*)malloc(sizeof(struct node));
    new->coef = coef;
    new->exp = exp;
    new->next = NULL;
    if(head1 == NULL)
        head1 = new;
    else {
        struct node *temp = head1;
        while(temp->next != NULL)
            temp = temp->next;
        temp->next = new;
    }
}
void insert2(int coef, int exp) {
    struct node *new = (struct node*)malloc(sizeof(struct node));
    new->coef = coef;
    new->exp = exp;
    new->next = NULL;
    if(head2 == NULL)
        head2 = new;
    else {
        struct node *temp = head2;
        while(temp->next != NULL)
            temp = temp->next;
        temp->next = new;
    }
}
void appendResult(int coef, int exp) {
    struct node *new = (struct node*)malloc(sizeof(struct node));
    new->coef = coef;
    new->exp = exp;
    new->next = NULL;
    if(head3 == NULL)
        head3 = new;
    else {
        struct node *temp = head3;
        while(temp->next != NULL)
            temp = temp->next;
        temp->next = new;
    }
}
```



```

}
}
void add() {
    struct node *temp1 = head1, *temp2 = head2;
    while(temp1 != NULL && temp2 != NULL) {
        if(temp1->exp == temp2->exp) {
            appendResult(temp1->coef + temp2->coef, temp1->exp);
            temp1 = temp1->next; temp2 = temp2->next;
        } else if(temp1->exp > temp2->exp) {
            appendResult(temp1->coef, temp1->exp);
            temp1 = temp1->next;
        } else {
            appendResult(temp2->coef, temp2->exp);
            temp2 = temp2->next;
        }
    }
    while(temp1 != NULL) { appendResult(temp1->coef, temp1->exp); temp1 = temp1->next; }
    while(temp2 != NULL) { appendResult(temp2->coef, temp2->exp); temp2 = temp2->next; }
}
void display(struct node *head) {
    struct node *temp = head;
    while(temp != NULL) {
        if(temp->next != NULL)
            printf("%d*x^%d + ", temp->coef, temp->exp);
        else
            printf("%d*x^%d", temp->coef, temp->exp);
        temp = temp->next;
    }
    printf("\n");
}
int main() {
    int n, i, coef, exp;
    printf("Enter the number of terms in the first polynomial: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("Enter the coefficient %d: ", i + 1);
        scanf("%d", &coef);
        printf("Enter the exponent %d: ", i + 1);
        scanf("%d", &exp);
        insert1(coef, exp);
    }
    printf("The first polynomial is:\n");
    display(head1);
    printf("Enter the number of terms in the second polynomial: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("Enter the coefficient %d: ", i + 1);
        scanf("%d", &coef);
        printf("Enter the exponent %d: ", i + 1);
        scanf("%d", &exp);
        insert2(coef, exp);
    }
}

```

```
printf("\nThe second polynomial is:\n");  
display(head2);  
add();  
printf("\nThe sum of the two polynomials is:\n");  
display(head3);  
return 0;  
}
```

Program 5: Linked list length even or odd

```
//Linked list length even or odd
#include
#include
struct node
{
    int data;
    struct node *next;
};
struct node *head=NULL;
int length(struct node *head)
{
    int cnt=0;
    while(head!=NULL)
    {
        cnt++;
        head=head->next;
    }
    return cnt;
}
int main()
{
    int i,n,data;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        struct node *newnode=(struct node*)malloc(sizeof(struct node));
        newnode->data=data;
        newnode->next=head;
        head=newnode;
    }
    int len=length(head);
    if(len%2==0)
    {
        printf("The length of the linked list is even: %d\n",len);
    }
    else
    {
        printf("The length of the linked list is odd: %d\n",len);
    }
}
```

Program 6: Delete a node in single linked list

```
//Delete a node in single linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
            temp=temp->n;
        temp->n=new;
    }
}
void delete(int data)
{
    struct node *temp=head;
    if(temp->d==data)
    {
        head=temp->n;
        free(temp);
    }
    else
    {
        struct node *temp1=head;
        while(temp1->n->d!=data)
            temp1=temp1->n;
        temp1->n=temp1->n->n;
        free(temp1->n);
    }
}
void display()
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
```

```
}
printf("\n");
}
int main()
{
    int n,data;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(int i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        insert(data);
    }
    display();
    printf("Enter the data of the node to be deleted: ");
    scanf("%d",&data);
    delete(data);
    display();
    return 0;
}
```

Program 7: First create a loop in a user defined linked list a at a given position and then detect wether it is looped or not

```
//First create a loop in a user defined linked list a at a given position and then
detect wether it is looped or not
//Detect loop in linked list
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(head==NULL)
{
head=new;
}
else
{
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=new;
}
}
void display()
{
struct node *temp=head;
int c=0;
while(temp!=NULL&& c<20)
{
printf("%d ",temp->d);
temp=temp->n;
c++;
}
if(temp!=NULL)
{
printf(" list is cyclic\n");
}
else
{
printf("\n");
}
}
void createLoop(int pos)
```

```

{
if(pos<=0)
return;
struct node *ls=head;
int c=1;
while(ls!=NULL&&c; {
ls=ls->n;
c++;
}
if(ls==NULL)
{
printf("Invalid position\n");
return;
}
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=ls;
printf("Loop created: Last node now points to node at position %d.\n", pos);
}
void detectloop()
{
struct node *slow=head;
struct node *fast=head;
int le=0;
while(fast!=NULL&&fast->n!=NULL)
{
slow=slow->n;
fast=fast->n->n;
if(slow==fast)
{
le=1;
break;
}
}
if(le!=1)
{
printf("No loop detected.\n");
return;
}
else
{
printf("Loop detected.\n");
}
}
int main()
{
int n,i,data,pos;
printf("Enter the number of nodes in the linked list: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter data for node %d: ",i+1);

```

```
scanf("%d",&data);  
insert(data);  
}  
printf("Linked list: ");  
display();  
printf("Enter the position to create a loop: ");  
scanf("%d",&pos);  
createLoop(pos);  
printf("The looped linked list:\n");  
display();  
detectloop();  
return 0;  
}
```


Program 8: count nodes of a singly linked list

```
//count nodes of a singly linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->n=NULL;
    if(head==NULL)
    {
        head=temp;
    }
    else
    {
        struct node *temp1=head;
        while(temp1->n!=NULL)
        {
            temp1=temp1->n;
        }
        temp1->n=temp;
    }
}
void display()
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
int count()
{
    int c=0;
    struct node *temp=head;
    while(temp!=NULL)
    {
        c++;
        temp=temp->n;
    }
    return c;
}
int main()
```

```
{  
int n,i,c;  
printf("Enter the number of nodes: ");  
scanf("%d",&n);  
for (i=0;i {  
int data;  
printf("Enter the data for node %d: ",i+1);  
scanf("%d",&data);  
insert(data);  
}  
display();  
c=count();  
printf("The number of nodes in the linked list is %d",c);  
return 0;  
}
```

Program 9: Print linked list elements

```
// Print linked list elements
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
            temp=temp->n;
        temp->n=new;
    }
}
void display()
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
int main()
{
    int n,data;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    for (int i=0;i {
        printf("Enter data for node %d: ",i+1);
        scanf("%d",&data);
        insert(data);
    }
    display();
    return 0;
}
```

Program 10: Reverse a doubly linked list

```
//Reverse a doubly linked list
#include
#include
struct node
{
    int d;
    struct node *p;
    struct node *n;
};
struct node *head=NULL;
struct node *insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    new->p=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
        {
            temp=temp->n;
        }
        temp->n=new;
        new->p=temp;
    }
    return head;
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
void reverse()
{
    struct node *prev=NULL, *curr=head, *next=NULL;
    while(curr!=NULL)
    {
        next=curr->n;
        curr->n=prev;
        curr->p=next;
    }
}
```

```
prev=curr;
curr=next;
}
head=prev;
}
int main()
{
    int n,i,data;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        insert(data);
    }
    printf("Original list: ");
    display(head);
    reverse();
    printf("Reversed list: ");
    display(head);
    return 0;
}
```

Program 11: take a user defined linked list take its alternative elements and than swap them and than reverse that linked list and than add it to the remaining of the original linked list elements that was not selected

//take a user defined linked list take its alternative elements and than swap them and than reverse that linked list and than add it to the remaining of the original linked list elements that was not selected

```
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(head==NULL)
{
head=new;
}
else
{
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=new;
}
}
void
```

Program 12: Add Two Numbers Represented as Linked Lists

```
//Add Two Numbers Represented as Linked Lists
//do it like a full adder
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *insert(struct node **head, int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    if(*head==NULL)
    {
        *head=new;
    }
    else
    {
        struct node *temp=*head;
        while(temp->n!=NULL)
        {
            temp=temp->n;
        }
        temp->n=new;
    }
    return *head;
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
struct node *rev(struct node *head)
{
    struct node *prev=NULL;
    struct node *curr=head;
    struct node *next=NULL;
    while(curr!=NULL)
    {
        next=curr->n;
        curr->n=prev;
        prev=curr;
        curr=next;
    }
}
```

```

}
return prev;
}
struct node *adder(struct node *head1, struct node *head2)
{
    struct node *curr1=head1;
    struct node *curr2=head2;
    struct node *head3=NULL;
    struct node *curr3=NULL;
    int carry=0;
    while(curr1!=NULL || curr2!=NULL || carry!=0)
    {
        int sum=0;
        if(curr1!=NULL)
        {
            sum+=curr1->d;
            curr1=curr1->n;
        }
        if(curr2!=NULL)
        {
            sum+=curr2->d;
            curr2=curr2->n;
        }
        sum+=carry;
        carry=sum/10;
        struct node *new=(struct node*)malloc(sizeof(struct node));
        new->d=sum%10;
        new->n=NULL;
        if(head3==NULL)
        {
            head3=new;
            curr3=new;
        }
        else
        {
            curr3->n=new;
            curr3=new;
        }
    }
    return head3;
}
int main()
{
    struct node *head1=NULL;
    struct node *head2=NULL;
    struct node *head3=NULL;
    int n1,n2,i,data;
    printf("Enter the number of nodes in the first linked list: ");
    scanf("%d",&n1);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
    }
}

```



```
insert(&head1;,data);
}
printf("Enter the number of nodes in the second linked list: ");
scanf("%d",&n2);
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
insert(&head2;,data);
}
head1=rev(head1);
head2=rev(head2);
head3=adder(head1,head2);
head3=rev(head3);
printf("First Number is: ");
display(rev(head1));
printf("Second Number is: ");
display(rev(head2));
printf("Sum Linked List: ");
display(head3);
return 0;
}
```

Program 13: Check if circular linked list

```
//Check if circular linked list
// Create a circular linked list and check if it is circular or not
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->n=NULL;
    if(head==NULL)
    {
        head=temp;
    }
    else
    {
        struct node *temp1=head;
        while(temp1->n!=NULL)
        {
            temp1=temp1->n;
        }
        temp1->n=temp;
    }
}
void cinsert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    if(head==NULL)
    {
        new->n=new;
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=head)
        {
            temp=temp->n;
        }
        temp->n=new;
        new->n=head;
    }
}
void display(struct node *head)
```

```

{
if(head==NULL)
return;
int c=0;
struct node *temp=head;
do
{
printf("%d ",temp->d);
temp=temp->n;
c++;
}
while(temp!=head&&c<20);
printf("\n");
}
void check()
{
if(head==NULL)
{
printf("The linked list is empty.\n");
return;
}
struct node *temp=head->n;
while(temp!=NULL&&temp!=head)
{
temp=temp->n;
}
if(temp==head)
printf("The linked list is circular.\n");
else
printf("The linked list is not circular.\n");
}
int main()
{
int n,i,data,choice;
printf("Enter the number of nodes in the linked list: ");
scanf("%d",&n);
printf("Choose 1 for circular linked list and 2 for non-circular linked list: ");
scanf("%d",&choice);
if(choice==1)
{
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
cinsert(data);
}
}
else if(choice==2)
{
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
insert(data);
}
}
}

```

```
}  
}  
else  
{  
printf("Invalid choice.\n");  
return 0;  
}  
printf("The linked list is: ");  
display(head);  
check();  
return 0;  
}
```

Program 14: insertion in a user defined linked list

```
//insertion in a user defined linked list
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
struct node *temp=(struct node*)malloc(sizeof(struct node));
temp->d=data;
temp->n=NULL;
if(head==NULL)
{
head=temp;
}
else
{
struct node *temp1=head;
while(temp1->n!=NULL)
temp1=temp1->n;
temp1->n=temp;
}
}
void display()
{
struct node *temp=head;
while(temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
}
int main()
{
int n,data,pos;
printf("Enter the number of nodes you want to create: ");
scanf("%d",&n);
for(int i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
insert(data);
}
printf("The linked list is: \n");
display();
printf("\nEnter the position where you want to insert the new node: ");
scanf("%d",&pos);
printf("Enter the data for the new node: ");
```

```
scanf("%d",&data);
struct node *temp=head;
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(pos<=0 || pos>n+1)
{
printf("Invalid position\n");
return 0;
}
else if(pos==1)
{
new->n=head;
head=new;
}
else
{
for(int i=1;i {
temp=temp->n;
}
new->n=temp->n;
temp->n=new;
}
printf("The linked list after insertion is: ");
display();
return 0;
}
```

Program 15: Find length of loop in linked list

```
//Find length of loop in linked list
//create a loop in a linked list at a given position and then find the length of the
loop
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(head==NULL)
{
head=new;
}
else
{
struct node *temp=head;
while(temp->n!=NULL)
{
temp=temp->n;
}
temp->n=new;
}
}
void display()
{
struct node *temp=head;
int c=0;
while(temp!=NULL && c<20)
{
printf("%d ",temp->d);
temp=temp->n;
c++;
}
if(temp!=NULL)
{
printf("---list is cyclic");
}
printf("\n");
}
void createLoop(int pos)
{
if(pos<=0)
{
```

```

return;
}
struct node *ls=head;
int c=1;
while(ls!=NULL && c {
ls=ls->n;
c++;
}
if(ls==NULL)
{
printf("Invalid position\n");
return;
}
struct node *temp=head;
while(temp->n!=NULL)
{
temp=temp->n;
}
temp->n=ls;
printf("Loop created: Last node now points to node at position %d.\n",pos);
}
int lengthOfloop()
{
struct node *slow=head;
struct node *fast=head;
int len=0;
while(fast!=NULL && fast->n!=NULL)
{
slow=slow->n;
fast=fast->n->n;
if(slow==fast)
{
struct node *ptr=slow;
do
{
len++;
ptr=ptr->n;
}
while(ptr!=slow);
return len;
}
}
return 0;
}
int main()
{
int n,i,data,pos,c;
printf("Enter the number of nodes in the linked list: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter data for node %d: ",i+1);
scanf("%d",&data);

```



```
insert(data);
}
printf("Linked list: ");
display();
printf("Enter the position to create a loop: ");
scanf("%d",&pos);
createLoop(pos);
printf("The looped linked list:\n");
display();
c=lengthOfloop();
printf("Length of the loop: %d\n",c);
return 0;
}
```

Program 16: Create a stack using linked list

```
//Create a stack using linked list
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void push(int data)
{
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=head;
head=new;
}
int pop()
{
if (head==NULL)
{
printf("Stack is empty\n");
return -1;
}
int data=head->d;
struct node *temp=head;
head=head->n;
free(temp);
return data;
}
void display()
{
struct node *temp=head;
while (temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
}
int main()
{
int choice,data;
while(1)
{
printf("1. Push\n");
printf("2. Pop\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
```

```
{
case 1:
printf("Enter the data to be pushed: ");
scanf("%d", &data);
push(data);
break;
case 2:
data = pop();
if (data != -1)
{
printf("Popped data: %d\n", data);
}
break;
case 3:
display();
printf("\n");
break;
case 4:
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Program 17: first make a loop out of the user defined linked list from the user defined point and then show it and then delete the loop

```
//first make a loop out of the user defined linked list from the user defined point and
then show it and then delete the loop
//Remove loop in linked list
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(head==NULL)
{
head=new;
}
else
{
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=new;
}
}
void display()
{
struct node *temp=head;
int c=0;
while(temp!=NULL&& c<20)
{
printf("%d ",temp->d);
temp=temp->n;
c++;
}
if(temp!=NULL)
{
printf("-----list is cyclic\n");
}
printf("\n");
}
void createLoop(int pos)
{
if(pos<=0)
return;
```

```

struct node *h=head;
int c=1;
while(h!=NULL&& c; {
h=h->n;
c++;
}
if(h==NULL)
{
printf("Invalid position\n");
return;
}
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=h;
printf("Loop created: Last node now points to node at position %d.\n", pos);
}
void rmloop()
{
struct node *slow=head;
struct node *fast=head;
int le=0;
while(fast!=NULL&&fast->n!=NULL)
{
slow=slow->n;
fast=fast->n->n;
if(slow==fast)
{
le=1;
break;
}
}
if(le!=1)
{
printf("No loop detected.\n");
return;
}
slow=head;
if(slow==fast)
{
while(fast->n!=slow)
fast=fast->n;
}
else
{
while(slow->n!=fast->n)
{
slow=slow->n;
fast=fast->n;
}
}
fast->n=NULL;

```

```
printf("The loop has been removed.\n");
}
int main()
{
    int n,i,data,pos;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter data for node %d: ",i+1);
        scanf("%d",&data);
        insert(data);
    }
    printf("Linked list: ");
    display();
    printf("Enter the position to create a loop: ");
    scanf("%d",&pos);
    createLoop(pos);
    printf("The looped linked list:\n");
    display();
    rmloop();
    printf("Linked list after loop removal: ");
    display();
    return 0;
}
```

Program 18: create a program to convert a string into a number and vice versa
example : three hundred = 300

```
//create a program to convert a string into a number and vice versa example : three hundred = 300
#include
#include
int main()
{
    int num;
    char str[10];
    printf("Enter a number: ");
    scanf("%d", &num);
    sprintf(str, "%d", num);
    printf("The string representation of %d is: %s\n", num, str);
    return 0;
}
```

Program 19: create a code to print this pattern-

```
//create a code to print this pattern-
//A
//b C
//D e F
//g H i J
#include
int main()
{
printf("Enter the number of rows: ");
int i, l;
scanf("%d", &l);
int j, k=65, n=1;
for(i=1; i<=l; i++)
{
for(j=1; j<=i; j++)
{
if(i==1 || i%2==0)
{
if(n%2==0)
{
printf("%c ", (k+32));
}
else
printf("%c ", k);
}
else
{
if(n%2!=0)
printf("%c ", k+32);
else
printf("%c ", k);
}
n++;
k++;
}
printf("\n");
}
return 0;
}
```


Program 20: make a program to convert infix expression to prefix expression with the help of stack

```
//make a program to convert infix expression to prefix expression with the help of
stack
/*#include
int stack[100],top=-1;
void push(int item)
{
stack[++top]=item;
}
int pop()
{
return stack[top--];
}
int precedence(char ch)
{
if(ch=='^')
return 3;
else if(ch=='*' || ch=='/')
return 2;
else if(ch=='+' || ch=='-')
return 1;
else
return 0;
}
int main()
{
char infix[100],prefix[100];
int i,j=0;
printf("Enter the infix expression: ");
scanf("%s",infix);
for(i=0;infix[i]!='\0';i++)
{
if(infix[i]=='(')
push(infix[i]);
else if(infix[i]==')')
{
while(stack[top]!='(')
{
prefix[j++]=pop();
}
top--;
}
else if(infix[i]=='+' || infix[i]=='-' || infix[i]=='*' || infix[i]=='/' || infix[i]=='^')
{
while(top!=-1&&stack[top]!='(' && precedence(infix[i])<=precedence(stack[top]))
{
prefix[j++]=pop();
}
push(infix[i]);
}
```

```

}
else
prefix[j++]=infix[i];
}
while(top!=-1)
{
prefix[j++]=pop();
}
prefix[j]='\0';
for(i=j-1;i>=0;i--)
{
printf("%c",prefix[i]);
}
return 0;
}*/
#include
#include
int stack[100],top=-1;
void push(int item)
{
stack[++top]=item;
}
int pop()
{
return stack[top--];
}
int precedence(char ch)
{
if(ch=='+'||ch=='-')
return 1;
else if(ch=='*'||ch=='/')
return 2;
else if(ch=='^')
return 3;
else
return 0;
}
void reverse(char *exp)
{
int i, len=strlen(exp);
for(i=0;i {
char temp=exp[i];
exp[i]=exp[len-i-1];
exp[len-i-1]=temp;
}
}
int main()
{
char exp[100],prefix[100];
int i,j=0;
printf("Enter the infix expression: ");
scanf("%s",exp);

```

```

reverse(exp);
for(i=0;exp[i]!='\0'; i++)
{
    if(exp[i]=='(')
        exp[i]=')';
    else if(exp[i]==')')
        exp[i]='(';
}
for(i=0;exp[i]!='\0';i++)
{
    if(exp[i]=='(')
        push(exp[i]);
    else if(exp[i]==')')
    {
        while(stack[top]!='(')
        {
            prefix[j++]=pop();
        }
        top--;
    }
    else if(exp[i]=='+'||exp[i]=='-'||exp[i]=='*'||exp[i]=='/'||exp[i]=='^')
    {
        while(top!=-1&&precedence;(exp[i])<=precedence(stack[top]))
        {
            prefix[j++]=pop();
        }
        push(exp[i]);
    }
    else
        prefix[j++] = exp[i];
}
while (top!=-1)
{
    prefix[j++]=pop();
}
prefix[j]='\0';
reverse(prefix);
printf("The prefix expression is: %s\n", prefix);
return 0;
}

```

Program 21: make a program to covert infix expression to postfix expression with the help of stack

```
// make a program to covert infix expression to postfix expression with the help of
stack
#include
int stack[100],top=-1;
void push(int item)
{
stack[++top]=item;
}
int pop()
{
return stack[top--];
}
int precedence(char ch)
{
if(ch=='+'||ch=='-')
return 1;
else if(ch=='*'||ch=='/')
return 2;
else if(ch=='^')
return 3;
else
return 0;
}
int main()
{
char exp[100],pos[100];
int i,j=0;
printf("Enter the infix expression: ");
scanf("%s",exp);
for(i=0;exp[i]!='\0';i++)
{
if(exp[i]=='(')
push(exp[i]);
else if(exp[i]==')')
{
while(stack[top]!='(')
{
pos[j++]=stack[top--];
}
top--;
}
else if(exp[i]=='+'||exp[i]=='-'||exp[i]=='*'||exp[i]=='/'||exp[i]=='^')
{
while(top!=-1&&precedence(exp[i])<=precedence(stack[top]))
{
pos[j++]=stack[top--];
}
push(exp[i]);
}
```

```
}  
else  
pos[j++]=exp[i];  
}  
while(top!=-1)  
{  
pos[j++]=stack[top--];  
}  
printf("The postfix expression is: %s",pos);  
return 0;  
}
```

Program 22: test.c

```
#include
#include

struct Node {
    int data;
    struct Node* next;
};

void insert(struct Node** head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void createLoop(struct Node* head, int position) {
    struct Node* temp = head;
    struct Node* loopNode = NULL;
    int count = 1;

    while (temp->next != NULL) {
        if (count == position)
            loopNode = temp;
        temp = temp->next;
        count++;
    }
    temp->next = loopNode;
}

int countLoopLength(struct Node* head) {
    struct Node *slow = head, *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow == fast) {
        int loopLength = 1;
        struct Node* temp = slow;
        while (temp->next != slow) {
```

```

loopLength++;
temp = temp->next;
}
return loopLength;
}
}
return 0;
}

int main() {
struct Node* head = NULL;
int n, value, position;

printf("Enter number of nodes: ");
scanf("%d", &n);

printf("Enter elements:\n");
for (int i = 0; i < n; i++) {
scanf("%d", &value);
insert(&head, value);
}

printf("Enter loop position (1-based index, 0 for no loop): ");
scanf("%d", &position);
if (position > 0)
createLoop(head, position);

int loopLength = countLoopLength(head);
if (loopLength > 0)
printf("Loop detected! Length of loop: %d\n", loopLength);
else
printf("No loop detected.\n");

return 0;
}

```

Program 23: DONE

```
//DONE
// make an code for deleting an element in an array
#include
int main()
{
    int n;
    printf("Enter the size of the array: ");
    scanf("%d",&n);
    int arr[n],i;
    printf("Enter the elements of the array: ");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    printf("Enter the position of the element to be deleted: ");
    scanf("%d",&i);
    for(int j=i-1;j {
        arr[j]=arr[j+1];
    }
    printf("The new array is: ");
    for(i=0;i {
        printf("%d ",arr[i]);
    }
    return 0;
}
```


Program 24: pointer to structure

```
//pointer to structure
#include
struct student
{
    char name[20];
    int roll;
    float marks;
};
int main()
{
    struct student s1;
    struct student *ptr=&s1;
    printf("Enter name of the student: ");
    scanf("%s", ptr->name);
    printf("Enter roll number of the student: ");
    scanf("%d", &ptr->roll);
    printf("Enter marks of the student: ");
    scanf("%f", &ptr->marks);
    printf("Name of the student: %s\n", ptr->name);
    printf("Roll number of the student: %d\n", ptr->roll);
    printf("Marks of the student: %f\n", ptr->marks);
    return 0;
}
```

Program 25: DONE

```
//DONE
// to insert an element in an array
#include
int main()
{
    int n,i,k,temp1,temp2,p;
    printf("Enter the size of the array:");
    scanf("%d",&n);
    int a[n];
    printf("Enter the elements of the array");
    for(i=0;i {
        scanf("%d",&a[i]);
    }
    printf("Enter the number to be inserted in the array:");
    scanf("%d",&k);
    printf("Enter the position to be inserted in the array;");
    scanf("%d",&p);
    for(i=p-1;i {
        temp1=a[i];
        if(i==p-1)
        {
            a[i]=k;
            temp2=a[i+1];
            a[i+1]=temp1;
        }
        else{
            temp1=temp2;
            temp2=a[i+1];
            a[i+1]=temp1;
        }
    }
    printf("The new array will be:");
    for(i=0;i {
        printf("%d ",a[i]);
    }
}
```

Program 26: DONE

```
//DONE
// find the sum of an array
#include
int main()
{
    int n;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    int arr[n],i,sum=0;
    printf("Enter the elements of the array:");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    for(i=0;i {
        sum=sum+arr[i];
    }

    {
        printf("The sum of the array elements is %d",sum);
    }
}
```

Program 27: DONE

```
//DONE
// print the elements of the array
#include
int main()
{
    int n;
    printf("Enter the size of the array:");
    scanf("%d",&n);
    int arr[n],i;
    printf("Enter the elements of the array:");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    for(i=0;i {
        printf("The elements of the array are %d\n",arr[i]);
    }
}
```

Program 28: DONE

```
//DONE
//product of an array with a single element
#include
int main()
{
    int n;
    printf("Enter the size of the array: ");
    scanf("%d",&n);
    int arr[n],i,p=1;
    printf("Enter the elements of the array");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    printf("Enter the element to be multiplied");
    scanf("%d",&p);
    for(i=0;i {
        arr[i]=arr[i]*p;
    }
    for(i=0;i {
        printf("%d ",arr[i]);
    }
    return 0;
}
```

Program 29: DONE

```
//DONE
//define a structure of employee having data members name,address,age and salary,take
data for an employee in an array dynamically and find the avg salary
#include
#include
struct employee
{
char name[20];
char address[20];
int age;
float salary;
};
struct employee *ptr;
int main()
{
int n,i;
printf("Enter the number of employees: ");
scanf("%d",&n);
ptr=(struct employee*)malloc(n*sizeof(struct employee));
for(i=0;i {
printf("Enter the name of employee %d: ",i+1);
scanf("%s",ptr[i].name);
printf("Enter the address of employee %d: ",i+1);
scanf("%s",ptr[i].address);
printf("Enter the age of employee %d: ",i+1);
scanf("%d",&ptr[i].age);
printf("Enter the salary of employee %d: ",i+1);
scanf("%f",&ptr[i].salary);
}
float sum=0;
for(i=0;i {
sum+=ptr[i].salary;
}
float avg=sum/n;
printf("The average salary of the employees is %f",avg);
free(ptr);
return 0;
}
```

Program 30: DONE

```
//DONE
//binary search
#include
int main()
{
    int a[10],i,n,k,l=0,h,m;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    printf("Enter the elements of the array\n");
    for(i=0;i {
        scanf("%d",&a[i]);
    }
    h=n-1;
    printf("Enter the element to be searched\n");
    scanf("%d",&k);
    while(l<=h)
    {
        m=(l+h)/2;
        if(k==a[m])
        {
            printf("Element found at position %d\n",m+1);
            break;
        }
        else if(k {
            h=m-1;
        }
        else
        {
            l=m+1;
        }
    }
    if(l>h)
    {
        printf("Element not found\n");
    }
    return 0;
}
```

Program 31: DONE

```
//DONE
// find the number of duplicate elements in the array
#include
int main()
{
    int n,i,j,count=0;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    int arr[n];
    printf("Enter the elements: ");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    for(i=0;i {
        for(j=i+1;j {
            if(arr[i]==arr[j])
            {
                count++;
            }
        }
    }
    printf("Number of duplicate elements: %d",count);
    return 0;
}
```


Program 32: DONE

```
//DONE
//insertion sort
#include
int main()
{
    int i,n,j,k=0;
    printf("Enter the size of the array");
    scanf("%d",&n);
    int arr[n];
    printf("Enter the elements of the array:");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    for(j=1;j {
        k=arr[j];
        i=j-1;
        while(i>=0&&arr[i]>k)
        {
            arr[i+1]=arr[i];
            i--;
        }
        arr[i+1]=k;
    }
    printf("The sorted array is:");
    for(i=0;i {
        printf("%d ",arr[i]);
    }
}
```

Program 33: DONE

```
//DONE
// program to create a dynamic array
#include
#include
int main()
{
    int *arr;
    int s;
    printf("Enter the size of the array: ");
    scanf("%d",&s);
    arr=(int*)malloc(s*sizeof(int));
    for (int i=0;i {
        printf("Enter element %d: ",i+1);
        scanf("%d",&arr[i]);
    }
    printf("Array elements are: ");
    for (int i=0;i {
        printf("%d ",arr[i]);
    }
    printf("\n");
    free(arr);
    return 0;
}
```

Program 34: DONE

```
//DONE
//find the median of an array
#include
int main()
{
    int n;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    int arr[n];
    printf("Enter the elements: ");
    for(int i=0;i {
        printf("Enter element %d: ",i+1);
        scanf("%d",&arr[i]);
    }
    int median;
    if(n%2==0)
    {
        median=(arr[n/2]+arr[n/2-1])/2;
    }
    else
    {
        median=arr[n/2];
    }
    printf("The median is %d",median);
    return 0;
}
```

Program 35: DONE

```
//DONE
//multiply two matrices
#include
int main()
{
    int m1,n1,m2,n2,i,j,k;
    printf("Enter the number of rows and columns of the first matrix: ");
    scanf("%dx%d",&m1,&n1);
    int a[m1][n1];
    printf("Enter the elements of the first matrix: \n");
    for(i=0;i {
        for(j=0;j {
            printf("A[%d][%d] = ",i+1,j+1);
            scanf("%d",&a[i][j]);
        }
    }
    printf("Enter the number of rows and columns of the second matrix: ");
    scanf("%dx%d",&m2,&n2);
    int b[m2][n2];
    printf("Enter the elements of the second matrix: \n");
    for(i=0;i {
        for(j=0;j {
            printf("B[%d][%d] = ",i+1,j+1);
            scanf("%d",&b[i][j]);
        }
    }
    if(n1!=m2)
    {
        printf("Matrices cannot be multiplied.\n");
        return 0;
    }
    int c[m1][n2];
    for(i=0;i {
        for(j=0;j {
            c[i][j]=0;
            for(k=0;k {
                c[i][j]+=a[i][k]*b[k][j];
            }
        }
    }
    printf("\nThe product of the matrices is:\n ");
    for(i=0;i {
        printf("\n\n");
        for(j=0;j {
            printf("%d ",c[i][j]);
        }
    }
    return 0;
}
```

Program 36: DONE

```
//DONE
//define structure of students having data members name, address and marks in Clanguage
, marks in information technology,take data for n students in an array dynamically and
find the total mars of 10
#include
#include
struct student
{
char name[20];
char address[20];
int clanguage;
int it;
};
int main()
{
int n;
printf("Enter the number of students: ");
scanf("%d",&n);
struct student *s=(struct student*)malloc(n*sizeof(struct student));
for(int i=0;i {
printf("Enter name of student %d: ",i+1);
scanf("%s",s[i].name);
printf("Enter address of student %d: ",i+1);
scanf("%s",s[i].address);
printf("Enter marks in clanguage of student %d: ",i+1);
scanf("%d",&s[i].clanguage);
printf("Enter marks in information technology of student %d: ",i+1);
scanf("%d",&s[i].it);
}
int sum=0;
for(int i=0;i {
sum=sum+s[i].clanguage+s[i].it;
}
printf("Total marks of %d students: %d\n",n,sum);
free(s);
return 0;
}
```

Program 37: DONE

```
//DONE
//addition of two matrices
#include
int main()
{ int m1,n1,i,m2,n2,j;
printf("Enter the number of rows and columns of the first matrix: ");
scanf("%d %d",&m1;,&n1;);
int a[m1][n1];
printf("Enter the elements of the first matrix:\n");
for(i=0;i {
for(j=0;j {
printf("Enter element A[%d][%d]: ",i+1,j+1);
scanf("%d",&a[i][j]);
}
}
printf("Enter the number of rows and columns of the second matrix: ");
scanf("%d %d",&m2;,&n2;);
int b[m2][n2];
printf("Enter the elements of the second matrix:\n");
for(i=0;i {
for(j=0;j {
printf("Enter element B[%d][%d]: ",i+1,j+1);
scanf("%d",&b[i][j]);
}
}
if(m1!=m2||n1!=n2)
{
printf("Matrices cannot be added.\n");
return 0;
}
int c[m1][n1];
for(i=0;i {
for(j=0;j {
c[i][j]=a[i][j]+b[i][j];
}
}
printf("matrix A:\n\n");
for(i=0;i {
for(j=0;j {
printf("%d ",a[i][j]);
}
printf("\n\n");
}
printf("matrix B:\n\n");
for(i=0;i {
for(int j=0;j {
printf("%d ",b[i][j]);
}
printf("\n\n");
}
```

```
printf("Sum of the matrices:\n\n");
for(i=0;i {
for(int j=0;j {
printf("%d ",c[i][j]);
}
printf("\n\n");
}
return 0;
}
```

Program 38: DONE

```
//DONE
//find max and min element in an array
#include
int main()
{
    int n;
    printf("Enter the size of the array: ");
    scanf("%d",&n);
    int a[n],i,max,min;
    for(i=0;i {
        printf("Enter element %d: ",i+1);
        scanf("%d",&a[i]);
    }
    max=a[0];
    min=a[0];
    for(i=1;i {
        if(a[i]>max)
        {
            max=a[i];
        }
        if(a[i] {
            min=a[i];
        }
    }
    printf("max=%d\nmin=%d",max,min);
}
```


Program 39: DONE

```
//DONE
// delete a node from a linked list after a node pointer x
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *create(int data)
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->n=NULL;
    return temp;
}
void display()
{
    struct node *temp=head;
    printf("Linked List: ");
    while (temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
int main()
{
    int n,i,pos,d;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&d);
        struct node *new=create(d);
        if (head==NULL)
        {
            head=new;
        }
        else
        {
            struct node *temp=head;
            while(temp->n!=NULL)
            {
                temp=temp->n;
            }
            temp->n=new;
        }
    }
```

```

}
}
display();
printf("Enter the position of x: ");
scanf("%d",&pos);
if (pos==0)
{
if(head==NULL)
{
printf("Linked List is empty\n");
return 0;
}
struct node *t=head;
t->n=t->n->n;
free(t);
}
else
{
if (head==NULL)
{
printf("Linked List is empty\n");
return 1;
}
struct node *temp=head;
for(i=0;in!=NULL;i++)
{
temp=temp->n;
}
struct node *t=temp->n;
temp->n=temp->n->n;
free(t);
}
display();
return 0;
}

```

Program 40: Represent and add two polynomials using linked lists

```
//Represent and add two polynomials using linked lists
#include
#include
struct node
{
    int coef;
    int exp;
    struct node *next;
};
struct node *head1=NULL;
struct node *head2=NULL;
struct node *head3=NULL;
void insert(struct node *head,int coef,int exp)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->coef=coef;
    new->exp=exp;
    new->next=NULL;
    if(head==NULL)
        head=new;
    else
    {
        struct node *temp=head;
        while(temp->next!=NULL)
        {
            temp=temp->next;
        }
        temp->next=new;
    }
}
// Helper function to append a new node to the result polynomial (head3).
void appendResult(struct node *head, int coef, int exp)
{
    struct node *new=(struct node *)malloc(sizeof(struct node));
    new->coef=coef;
    new->exp=exp;
    new->next=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp = head;
        while (temp->next!=NULL)
        {
            temp=temp->next;
        }
        temp->next=new;
    }
}
```

```

}
void add()
{
    struct node *temp1=head1;
    struct node *temp2=head2;
    while(temp1!=NULL&&temp2!=NULL)
    {
        if(temp1->exp==temp2->exp)
        {
            appendResult(head3,temp1->coef+temp2->coef,temp1->exp);
            temp1=temp1->next;
            temp2=temp2->next;
        }
        else if(temp1->exp>temp2->exp)
        {
            appendResult(head3,temp1->coef,temp1->exp);
            temp1=temp1->next;
        }
        else
        {
            appendResult(head3,temp2->coef,temp2->exp);
            temp2=temp2->next;
        }
    }
    while(temp1!=NULL)
    {
        appendResult(head3, temp1->coef, temp1->exp);
        temp1=temp1->next;
    }
    while(temp2!=NULL)
    {
        appendResult(head3,temp2->coef,temp2->exp);
        temp2=temp2->next;
    }
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        if(temp->next!=NULL)
        {
            printf("%d*x^%d + ",temp->coef,temp->exp);
            temp=temp->next;
        }
        else
        {
            printf("%d*x^%d\n",temp->coef,temp->exp);
        }
    }
}
int main()

```

```

{
    int n,i,coef,exp;
    printf("Enter the number of terms in the first polynomial: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the coefficient %d: ",i+1);
        scanf("%d",&coef);
        printf("Enter the exponent %d: ",i+1);
        scanf("%d",&exp);
        insert(head1,coef,exp);
    }
    printf("The first polynomial is: \n");
    display(head1);
    printf("Enter the number of terms in the second polynomial: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the coefficient %d: ",i+1);
        scanf("%d",&coef);
        printf("Enter the exponent %d: ",i+1);
        scanf("%d",&exp);
        insert(head2,coef,exp);
    }
    printf("\nThe second polynomial is: ");
    display(head2);
    add();
    printf("\nThe sum of the two polynomials is: ");
    display(head3);
    return 0;
}

```

Program 41: DONE

```
//DONE
// delete a node from a linked list before a node pointer x
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void create(int data)
{
    struct node *newnode=(struct node *)malloc(sizeof(struct node));
    newnode->d=data;
    newnode->n=NULL;
    if(head==NULL)
    {
        head=newnode;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
        {
            temp=temp->n;
        }
        temp->n=newnode;
    }
}
void delete(struct node *head,int data)
{
    if(head==NULL)
    {
        printf("List is empty");
        return;
    }
    // if head node is to be deleted
    if(head->d==data)
    {
        struct node *temp=head;
        head=head->n;
        free(temp);
        return;
    }
    struct node *temp=head, *prev=NULL;
    while(temp!=NULL && temp->d!=data)
    {
        prev=temp;
        temp=temp->n;
    }
```

```

if(temp==NULL)
{
printf("Data not found");
return;
}
prev->n=temp->n;
free(temp);
}
void display()
{
if(head==NULL)
{
printf("List is empty");
return;
}
struct node *temp=head;
while(temp!=NULL)
{
printf("%d->", temp->d);
temp=temp->n;
}
printf("NULL\n");
}
int main()
{
int ch,data;
while(1)
{
printf("1.Create\n2.Delete\n3.Display\n4.Exit\nEnter your choice: ");
scanf("%d",&ch);
switch(ch)
{
case 1:
printf("Enter the data: ");
scanf("%d",&data);
create(data);
break;
case 2:
printf("Enter the data to be deleted: ");
scanf("%d",&data);
delete(head,data);
break;
case 3:
display();
break;
case 4:
return 0;
default:
printf("Invalid choice\n");
}
}
}

```


Program 42: done

```
//done
// check wether a link is circular at any point or not
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;

void insert(int data)
{
    struct node *newnode=(struct node *)malloc(sizeof(struct node));
    newnode->d=data;
    newnode->n=head;
    head=newnode;
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
int isCircular(struct node *head)
{
    struct node *temp=head;
    while(temp->n!=NULL)
    {
        temp=temp->n;
    }
    if(temp==head)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
int check(struct node *head)
{
    if(isCircular(head))
    {
        printf("The linked list is circular\n");
        return 0;
    }
}
```

```
}
else
{
printf("The linked list is not circular\n");
return 0;
}
}
int main()
{
int n,i,data;
printf("Enter the number of nodes you want to create: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter the data for node %d: ", i + 1);
scanf("%d", &data);
insert(data);
}
display(head);
if(check(head))
{
printf("The linked list is circular.\n");
}
else
{
printf("The linked list is not circular.\n");
}
return 0;
}
```

Program 43: done

```
//done
// create a program to delete a node from a singly linked list after a node pointer x
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *create(int data)
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->n=NULL;
    return temp;
}
int main()
{
    int n,i,pos;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&pos);
        struct node *new=create(pos);
        if(head==NULL)
        {
            head=new;
        }
        else
        {
            struct node *temp;
            temp=head;
            while(temp->n!=NULL)
            {
                temp=temp->n;
            }
            temp->n=new;
        }
    }
    printf("Original Linked List: ");
    struct node *temp;
    temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
```

```
printf("\n");
int pos1;
printf("Enter the position of the node to be deleted: ");
scanf("%d",&pos1);
if(pos1==1)
{
head=head->n;
free(head);
}
else
{
struct node *temp;
temp=head;
for(i=1;i {
temp=temp->n;
}
temp->n=temp->n->n;
free(temp->n);
}
printf("Updated Linked List: ");
temp=head;
while(temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
printf("\n");
return 0;
}
```

Program 44: done

```
//done
//deletion of the middle element of a user defined linked list
#include
#include
struct node
{
    int data;
    struct node *next;
};
struct node *head=NULL;
void deleteMiddle(struct node *head)
{
    struct node *slow_ptr = head;
    struct node *fast_ptr = head;
    struct node *prev = NULL;
    while (fast_ptr != NULL && fast_ptr->next != NULL)
    {
        fast_ptr=fast_ptr->next->next;
        prev=slow_ptr;
        slow_ptr=slow_ptr->next;
    }
    prev->next=slow_ptr->next;
    free(slow_ptr);
}
int main()
{
    struct node *newnode;
    struct node *temp;
    int i,n,data;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data of node %d: ",i+1);
        scanf("%d",&data);
        newnode=(struct node*)malloc(sizeof(struct node));
        newnode->data=data;
        newnode->next=head;
        head=newnode;
    }
    temp=head;
    printf("\nLinked list before deletion: ");
    while(temp!=NULL)
    {
        printf("%d ",temp->data);
        temp=temp->next;
    }
    deleteMiddle(head);
    temp=head;
    printf("\nLinked list after deletion: ");
    while(temp!=NULL)
```

```
{  
printf("%d ",temp->data);  
temp=temp->next;  
}  
return 0;  
}
```

Program 45: delete a node from a circular linked list at any given position

//delete a node from a circular linked list at any given position

```
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=head)
        {
            temp=temp->n;
        }
        temp->n=new;
        new->n=head;
    }
    return head;
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
struct node *delete(struct node *head,int pos)
{
    struct node *temp=head;
    struct node *prev=NULL;
    if(pos==1)
    {
        return head;
    }
    else
```

```
{
for(int i=1;i {
prev=temp;
temp=temp->n;
}
prev->n=temp->n;
free(temp);
return head;
}
}
int main()
{
int data,pos,i,n;
printf("Enter the number of nodes: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter the data: ");
scanf("%d",&data);
insert(data);
}
display(head);
printf("\nEnter the position: ");
scanf("%d",&pos);
head=delete(head,pos);
display();
return 0;
}
```


Program 46: done

```
//done
// you are given a node of a singly linked list where you have to delete a value of the
given node from the linked list but you are not given the head of the list
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void display(struct node *head)
{
struct node *temp=head;
while(temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
}
int main()
{
struct node *new, *temp, *prev;
int i,n,data;
printf("Enter the number of nodes you want to create: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(head==NULL)
{
head=new;
prev=new;
}
else
{
prev->n=new;
prev=new;
}
}
display(head);
printf("\nEnter the position of the node you want to delete: ");
scanf("%d",&i);
temp=head;
if(i==1)
{
head=temp->n;
```

```

free(temp);
}
else
{
    int count=1;
    while(count {
        temp=temp->n;
        count++;
    }
    prev=temp;
    temp=temp->n;
    prev->n=temp->n;
    free(temp);
}
display(head);
printf("\nEnter the data of the node you want to delete: ");
scanf("%d",&data);
temp=head;
if(temp->d==data)
{
    head=temp->n;
    free(temp);
}
else
{
    while(temp->n!=NULL)
    {
        if(temp->n->d==data)
        {
            prev=temp;
            temp=temp->n;
            prev->n=temp->n;
            free(temp);
        }
    }
}
}
}
}

```

Program 47: done

```
//done
//delete a node from a user defined doubly linked list at any given position
#include
#include
struct node
{
    int d;
    struct node *p;
    struct node *n;
};
struct node *head=NULL;
void display(struct node *head)
{
    struct node *temp;
    temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
void insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->p=NULL;
    new->n=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
        {
            temp=temp->n;
        }
        temp->n=new;
        new->p=temp;
    }
}
void del(int pos)
{
    if(head==NULL)
    {
        printf("List is empty, nothing to delete.\n");
        return;
    }
    struct node *temp=head, *prev=NULL;
```

```

if(pos==1)
{
head=head->n;
free(temp);
return;
}
for(int i=1;temp!=NULL && i {
prev=temp;
temp=temp->n;
}
if(temp==NULL)
{
printf("Invalid position!\n");
return;
}
prev->n=temp->n;
free(temp);
}
int main()
{
int n,data,pos;
printf("Enter the number of nodes you want to create: ");
scanf("%d",&n);
for(int i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
insert(data);
}
display(head);
printf("\nEnter the position of the node you want to delete: ");
scanf("%d",&pos);
del(pos);
display(head);
return 0;
}

```

Program 48: done

```
//done
//count the number of each duplicated elements in a linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void create(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    if(head==NULL)
    {
        head=new;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
        {
            temp=temp->n;
        }
        temp->n=new;
    }
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
int count (struct node *head,int data)
{
    int c=0;
    struct node *temp=head;
    while(temp!=NULL)
    {
        if(temp->d==data)
        {
            c++;
        }
        temp=temp->n;
    }
}
```

```

return c;
}
int main()
{
    int i,n,data;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        create(data);
    }
    display(head);
    int cnt,d,temp=0;
    while(head!=NULL)
    {
        d=head->d;
        if(temp!=d)
        {
            cnt=count(head,d);
            if(cnt>1)
            {
                printf("\n%d is repeated %d times\n",d,cnt);
            }
            temp=d;
        }
        head=head->n;
    }
    printf("\n");
    return 0;
}

```

Program 49: done

```
//done
//remove duplicates from a linked list
#include
#include
struct node
{
    int data;
    struct node *next;
};
struct node *head=NULL;
void create(int data)
{
    struct node *new=(struct node *)malloc(sizeof(struct node));
    new->data=data;
    new->next=head;
    head=new;
}
void display()
{
    struct node *ptr=head;
    while(ptr!=NULL)
    {
        printf("%d ", ptr->data);
        ptr=ptr->next;
    }
}
void remove_duplicates()
{
    struct node *curr=head;
    struct node *run,*temp;
    while(curr!=NULL)
    {
        run=curr;
        while(run->next!=NULL)
        {
            if(run->next->data==curr->data)
            {
                temp=run->next;
                run->next=run->next->next;
                free(temp);
            }
            else
            {
                run=run->next;
            }
        }
        curr=curr->next;
    }
}
int main()
```

```
{
int n,data;
printf("Enter the number of nodes in the linked list: ");
scanf("%d",&n);
for(int i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
create(data);
}
printf("Original linked list: ");
display();
remove_duplicates();
printf("\nLinked list after removing duplicates: ");
display();
return 0;
}
```


Program 50: done

```
//done
//find the middle of linked list
#include
#include
struct node
{
    int data;
    struct node *next;
};
struct node *head=NULL;

void create(int data)
{
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    temp->data=data;
    temp->next=head;
    head=temp;
}
void display()
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->data);
        temp=temp->next;
    }
}
int find_middle()
{
    struct node *slow=head;
    struct node *fast=head;
    while (fast!=NULL&&fast->next!=NULL)
    {
        slow=slow->next;
        fast=fast->next->next;
    }
    return slow->data;
}
int main()
{
    int n , i;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(i=0;i {
        int data;
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        create(data);
    }
```

```
display();  
int x=find_middle();  
printf("\nMiddle element of the linked list is %d",x);  
return 0;  
}
```

Program 51: done

```
//done
//insert a node at any point given by the user in the linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
int main()
{
    struct node *head=NULL;
    struct node *second=NULL;
    struct node *third=NULL;
    struct node *fourth=NULL;
    struct node *fifth=NULL;
    head=(struct node *)malloc(sizeof(struct node));
    second=(struct node *)malloc(sizeof(struct node));
    third=(struct node *)malloc(sizeof(struct node));
    fourth=(struct node *)malloc(sizeof(struct node));
    fifth=(struct node *)malloc(sizeof(struct node));
    head->d=1;
    head->n=second;
    second->d=2;
    second->n=third;
    third->d=3;
    third->n=fourth;
    fourth->d=4;
    fourth->n=fifth;
    fifth->d=5;
    fifth->n=NULL;
    struct node *ptr=head;
    while (ptr!=NULL)
    {
        printf("%d ", ptr->d);
        ptr=ptr->n;
    }
    struct node *newnode=(struct node *)malloc(sizeof(struct node));
    int pos;
    printf("\nEnter the position at which you want to insert the node: ");
    scanf("%d",&pos);
    int n1;
    printf("\nEnter the value of the node: ");
    scanf("%d",&n1);
    if (pos==1)
    {
        newnode->d=6;
        newnode->n=head;
        head=newnode;
    }
```

```
else
{
ptr=head;
for(int i=1;i {
if (ptr->n==NULL)
{
printf("Invalid position\n");
return 0;
}
ptr=ptr->n;
}
newnode->d=n1;
newnode->n=ptr->n;
ptr->n=newnode;
}
ptr=head;
while(ptr!=NULL)
{
printf("%d ", ptr->d);
ptr=ptr->n;
}
return 0;
}
```

Program 52: done

```
//done
// insert a node at any given position for a user defined doubly linked list
#include
#include
struct node
{
    int d;
    struct node *p;
    struct node *n;
};
struct node *head=NULL;
struct node *create(int data)
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->p=NULL;
    temp->n=NULL;
    return temp;
}
void display(struct node *head)
{
    struct node *temp;
    temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
void insert(struct node *head,int pos,int d)
{
    struct node *new=create(d);
    struct node *temp=head;
    if(pos==1)
    {
        new->n=head;
        head->p=new;
        head=new;
    }
    else
    {
        int i=1;
        while(i {
            temp=temp->n;
            i++;
        }
        if(temp->n==NULL)
        {
```

```

temp->n=new;
new->p=temp;
}
else
{
new->n=temp->n;
new->p=temp;
temp->n=new;
new->n->p=new;
}
}
}
int main()
{
int i,n,pos,d;
printf("Enter the number of nodes in the linked list: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&d);
struct node *new=create(d);
if(head==NULL)
{
head=new;
}
else
{
struct node *temp=head;
while(temp->n!=NULL)
{
temp=temp->n;
}
temp->n=new;
new->p=temp;
}
}
display(head);
printf("Enter the position to be inserted: ");
scanf("%d",&pos);
printf("Enter the data to be inserted: ");
scanf("%d",&d);
insert(head,pos,d);
display(head);
return 0;
}

```

Program 53: done

```
//done
// insert a user defined node in the middle of a user defined singly linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *insert_node(struct node *head, int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=head;
    head=new;
    return head;
}
int find_middle(struct node *head)
{
    struct node *slow=head;
    struct node *fast=head;
    while (fast!=NULL&&fast->n!=NULL)
    {
        slow=slow->n;
        fast=fast->n->n;
    }
    return slow->d;
}
struct node *insert_middle(struct node *head, int data, int pos)
{
    struct node *new_node=(struct node*)malloc(sizeof(struct node));
    struct node *temp=head;
    new_node->d=data;
    for(int i=0;i {
        temp=temp->n;
    }
    new_node->n=temp->n;
    temp->n=new_node;
    return head;
}
int main()
{
    int n, data;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    for (int i=0;i {
        printf("Enter data for node %d: ",i+1);
        scanf("%d",&data);
        head=insert_node(head,data);
    }
```

```
}
struct node *temp=head;
while (temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
int pos;
pos=find_middle(head);
printf("\n Enter the data to be inserted in the middle: ");
scanf("%d",&data);
head=insert_middle(head,data,pos);
temp=head;
while(temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
printf("\n");
return 0;
}
```


Program 54: done

```
//done
// to insert a node in a linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *insert(struct node *head,int data)
{
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->d=data;
    newnode->n=NULL;
    if(head==NULL)
    {
        head=newnode;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=NULL)
        {
            temp=temp->n;
        }
        temp->n=newnode;
    }
    return head;
}
void display(struct node *head)
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
    printf("\n");
}
int main()
{
    int i,n,data;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        head=insert(head,data);
    }
```

```
struct node *temp=head;
printf("The linked list is: ");
display(temp);
printf("Enter the data to be inserted: ");
scanf("%d",&data);
head=insert(head,data);
temp=head;
printf("The linked list after insertion is: ");
display(temp);
return 0;
}
```

Program 55: done

```
//done
// Find the address of last node of the linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
int main()
{
    struct node *first=NULL;
    struct node *second=NULL;
    struct node *third=NULL;
    first=(struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));
    first->d=1;
    first->n=second;
    second->d=2;
    second->n=third;
    third->d=3;
    third->n=NULL;
    struct node *ptr=first;
    while(ptr->n!=NULL)
    {
        ptr=ptr->n;
    }
    printf("Address of last node: %p\n",ptr);
    return 0;
}
```

Program 56: done

```
//done
// a program to count the size of the linked list wether it is even or odd
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void create( int data )
{
struct node *newnode;
newnode=(struct node*)malloc(sizeof(struct node));
newnode->d=data;
newnode->n=head;
head=newnode;
}
void display(struct node *head)
{
struct node *temp;
temp=head;
while(temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
}
int main()
{
int i,n,data;
printf("Enter the number of nodes: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
create(data);
}
display(head);
int cnt=0;
struct node *temp=head;
while(temp!=NULL)
{
cnt++;
temp=temp->n;
}
printf("\nLength of linked list is %d and it is %s\n",cnt,(cnt%2==0)?"even":"odd");
return 0;
}
```

Program 57: done

```
//done
// to reverse a singly linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *reverse(struct node *head)
{
    struct node *prev=NULL;
    struct node *curr=head;
    struct node *next=NULL;
    while (curr!=NULL)
    {
        next=curr->n;
        curr->n=prev;
        prev=curr;
        curr=next;
    }
    head=prev;
    return head;
}
void display(struct node *head)
{
    struct node *temp=head;
    while (temp!=NULL)
    {
        printf("%d ",temp->d);
        temp=temp->n;
    }
}
int main()
{
    int n;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    for (int i=0;i {
        struct node *new=(struct node *)malloc(sizeof(struct node));
        printf("Enter the data for node %d: ", i + 1);
        scanf("%d",&new->d);
        new->n=head;
        head=new;
    }
    printf("Original Linked List: ");
    display(head);
    printf("\nReversed Linked List: ");
    head=reverse(head);
```

```
display(head);  
return 0;  
}
```

Program 58: done

```
//done
// sort and merge two linked lists
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head1=NULL;
struct node *head2=NULL;
struct node *head3=NULL;
void push(struct node **head, int new_data)
{
    struct node *new_node = (struct node*)malloc(sizeof(struct node));
    new_node->d = new_data;
    new_node->n = (*head);
    (*head) = new_node;
}
void display(struct node *head)
{
    while(head!=NULL)
    {
        printf("%d ", head->d);
        head=head->n;
    }
}
struct node *merge(struct node *head1, struct node *head2)
{
    struct node *p1=head1;
    struct node *p2=head2;
    struct node *p3=head3;
    while(p1!=NULL && p2!=NULL)
    {
        if(p1->d<p2->d)
        {
            p3->d=p1->d;
            p3->n=NULL;
            p1=p1->n;
            p3=p3->n;
        }
        else
        {
            p3->d=p2->d;
            p3->n=NULL;
            p2=p2->n;
            p3=p3->n;
        }
    }
    while(p1!=NULL)
```

```

{
p3->d=p1->d;
p3->n=NULL;
p1=p1->n;
p3=p3->n;
}
while(p2!=NULL)
{
p3->d=p2->d;
p3->n=NULL;
p2=p2->n;
p3=p3->n;
}
return head3;
}
int main()
{
int i,n;
printf("Enter the number of nodes in the first linked list: ");
scanf("%d",&n);
for(i=0;i {
int data;
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
push(&head1,,data);
}
printf("Enter the number of nodes in the second linked list: ");
scanf("%d",&n);
for(i=0;i {
int data;
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
push(&head2,,data);
}
printf("First Linked List: ");
display(head1);
printf("\nSecond Linked List: ");
display(head2);
printf("\nMerged Linked List: ");
display(merge(head1,head2));
return 0;
}

```


Program 59: done

```
//done
//Transverse the linked list
#include
#include
struct node
{
int d;
struct node *n;
};
int main()
{
struct node *first=NULL;
struct node *second=NULL;
struct node *third=NULL;
first=(struct node *)malloc(sizeof(struct node));
second=(struct node *)malloc(sizeof(struct node));
third=(struct node *)malloc(sizeof(struct node));
first->d=1;
first->n=second;
second->d=2;
second->n=third;
third->d=3;
third->n=NULL;
struct node *ptr=first;
while(ptr!=NULL)
{
printf("%d ", ptr->d);
ptr=ptr->n;
}
return 0;
}
```

Program 60: done

```
//done
// create case for insertion at beginning and at last of a circular linked list
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->d=data;
    newnode->n=NULL;
    if(head==NULL)
    {
        head=newnode;
        newnode->n=newnode;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=head)
        {
            temp=temp->n;
        }
        temp->n=newnode;
        newnode->n=head;
    }
}
void insert_at_beginning(int data)
{
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->d=data;
    newnode->n=NULL;
    if(head==NULL)
    {
        head=newnode;
        newnode->n=newnode;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=head)
        {
            temp=temp->n;
        }
        newnode->n=head;
        temp->n=newnode;
    }
}
```

```

head=newnode;
}
}
void insert_at_last(int data)
{
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    newnode->d=data;
    newnode->n=NULL;
    if(head==NULL)
    {
        head=newnode;
        newnode->n=newnode;
    }
    else
    {
        struct node *temp=head;
        while(temp->n!=head)
        {
            temp=temp->n;
        }
        temp->n=newnode;
        newnode->n=head;
    }
}
void display()
{
    if(head==NULL)
    {
        return;
    }
    struct node *temp=head;
    int count=0;
    do
    {
        printf("%d ",temp->d);
        temp=temp->n;
        count++;
    }
    while(temp!=head && count<20);
    printf("\n");
}
int main()
{
    int n,i,data,choice;
    printf("Enter the number of nodes to create: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        insert(data);
    }
    printf("The circular linked list is: ");

```

```
display();
printf("Enter 1 to insert at beginning, 2 to insert at last: ");
scanf("%d",&choice);
printf("Enter the data for the new node: ");
scanf("%d",&data);
if(choice==1)
{
insert_at_beginning(data);
}
else if(choice==2)
{
insert_at_last(data);
}
else
{
printf("Invalid choice.\n");
return 0;
}
printf("The updated circular linked list is: ");
display();
return 0;
}
```

Program 61: done

```
//done
//Create two stack in a single array
#include
int stack[100];
int top1=-1;
int top2=100;
void push1(int x)
{
    if(top1+1 {
        top1++;
        stack[top1]=x;
    }
    else
    {
        printf("Stack overflow for stack1\n");
    }
}
void push2(int x)
{
    if(top2-1>top1)
    {
        top2--;
        stack[top2]=x;
    }
    else
    {
        printf("Stack overflow for stack2\n");
    }
}
void display1()
{
    int i;
    printf("Stack 1:\n");
    for(i=top1;i>=0;i--)
    {
        printf("%d\n", stack[i]);
    }
}
void display2()
{
    int i;
    printf("Stack 2:\n");
    for(i=top2;i<100;i++)
    {
        printf("%d\n", stack[i]);
    }
}
int main()
{
    int n1, n2, d;
```

```
printf("Enter the number of elements to push into stack1: ");
scanf("%d", &n1);
for(int i=0;i {
printf("Enter element for stack1, number %d: ",i+1);
scanf("%d",&d);
push1(d);
}
printf("Enter the number of elements to push into stack2: ");
scanf("%d",&n2);
for(int i=0;i {
printf("Enter element for stack2, number %d: ",i+1);
scanf("%d", &d);
push2(d);
}
display1();
display2();
return 0;
}
```

Program 62: DONE

```
//DONE
//implement queue using stack and perform enqueue dequeue and display on it
#include
int s1[100];
int s2[100];
int top1=-1;
int top2=-1;
void enqueue(int x)
{
if(top1==-1)
{
top1=0;
s1[top1]=x;
}
else
{
s1[++top1]=x;
}
}
void dequeue()
{
if(top1==-1)
{
printf("Queue is empty\n");
return;
}
else
{
if(top2==-1)
{
while(top1>=0)
{
s2[++top2]=s1[top1--];
}
printf("Dequeued element is %d\n",s2[top2--]);
while(top2>=0)
{
s1[++top1]=s2[top2--];
}
}
}
}
void display()
{
if(top1==-1)
{
printf("Queue is empty\n");
}
else
{

```

```
for(int i=0;i<=top1;i++)
{
printf("%d ",s1[i]);
}
printf("\n");
}
}
int main()
{
int data;
while(1)
{
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
int choice;
scanf("%d",&choice);
switch(choice)
{
case 1:
printf("Enter the data to enqueue: ");
scanf("%d",&data);
enqueue(data);
break;
case 2:
dequeue();
break;
case 3:
display();
break;
case 4:
return 0;
default:
printf("Invalid choice\n");
}
}
return 0;
}
```


Program 63: DONE

```
//DONE
//implementation of queue through dynamic array and perform enqueue dequeue and display
on it
#include
#include
int *q;
int f=-1;
int r=-1;
void enq(int n)
{
    int item;
    printf("Enter the item to be inserted:");
    scanf("%d", &item);
    if(f== -1&&r== -1)
    {
        f=r=0;
        q[r]=item;
    }
    else
    {
        if(f==0&&r==n-1)
        {
            printf("Queue is full\n");
        }
        else
        {
            r++;
            q[r]=item;
        }
    }
}
void deq()
{
    if(f== -1&&r== -1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        if (f==r)
        {
            f=r=-1;
        }
        else
        {
            printf("Deleted item is:%d\n",q[f]);
            f++;
        }
    }
}
```

```

void display()
{
    if (f== -1 && r== -1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Queue elements are:\n");
        for (int i=f; i<=r; i++)
        {
            printf("%d ", q[i]);
        }
        printf("\n");
    }
}

int main()
{
    int choice, n;
    printf("Enter the size of the queue:");
    scanf("%d", &n);
    q=(int*)malloc(n*sizeof(int));
    f=r=-1;
    while (1)
    {
        printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
        printf("Enter your choice:");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                enq(n);
                break;
            case 2:
                deq();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}

```

Program 64: DONE

```
//DONE
//implement queue through a linked list and perform enqueue dequeue and display on it
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
struct node *newnode(int data)
{
    struct node *temp;
    temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->n=NULL;
    return temp;
}
void enqueue(int data)
{
    struct node *temp=newnode(data);
    if(head==NULL)
    {
        head=temp;
    }
    else
    {
        struct node *temp1=head;
        while(temp1->n!=NULL)
        {
            temp1=temp1->n;
        }
        temp1->n=temp;
    }
}
void dequeue()
{
    if(head==NULL)
    {
        printf("Queue is empty\n");
    }
    else
    {
        struct node *temp=head;
        head=head->n;
        printf("Dequeued element is %d\n",temp->d);
        free(temp);
    }
}
void display()
```

```

{
if(head==NULL)
{
printf("Queue is empty");
}
struct node *temp=head;
while(temp!=NULL)
{
printf("%d ",temp->d);
temp=temp->n;
}
printf("\n");
}
int main()
{
int choice,data;
while(1)
{
printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
printf("Enter your choice: ");
scanf("%d",&choice);
switch(choice)
{
case 1:
printf("Enter the data to enqueue: ");
scanf("%d",&data);
enqueue(data);
break;
case 2:
dequeue();
break;
case 3:
display();
break;
case 4:
return 0;
default:
printf("Invalid choice\n");
}
}
return 0;
}

```

Program 65: DONE

```
//DONE
//implement queue through an array and perform enqueue dequeue and display on it
#include
int q[100];
int f=-1;
int r=-1;
void enq()
{
    int item;
    printf("Enter the item to be inserted:");
    scanf("%d",&item);
    if(f== -1 && r== -1)
    {
        f=r=0;
        q[r]=item;
    }
    else
    {
        if(f==0 && r==99)
        {
            printf("Queue is full");
        }
        else
        {
            r++;
            q[r]=item;
        }
    }
}
void deq()
{
    if(f== -1 && r== -1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        printf("Deleted item is:%d\n",q[f]);
        f++;
        if(f>r)
        {
            f=r=-1;
        }
    }
}
void display()
{
    if(f== -1 && r== -1)
    {
        printf("Queue is empty\n");
    }
}
```

```
}  
else  
{  
for(int i=f;i<=r;i++)  
{  
printf("%d ",q[i]);  
}  
printf("\n");  
}  
}  
int main()  
{  
int choice;  
while(1)  
{  
printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");  
printf("Enter your choice:");  
scanf("%d",&choice);  
switch(choice)  
{  
case 1:  
enq();  
break;  
case 2:  
deq();  
break;  
case 3:  
display();  
break;  
case 4:  
return 0;  
default:  
printf("Invalid choice\n");  
}  
}  
}
```

Program 66: implement stack using 2 queues and perform push pop and display on it

```
//implement stack using 2 queues and perform push pop and display on it
#include
int q1[100],q2[100];
int f1=-1,r1=-1;
int f2=-1,r2=-1;
void push(int x)
{
    if(f1==-1&&r1==-1)
    {
        f1=r1=0;
        q1[r1]=x;
    }
    else
    {
        f2=0;
        r2=0;
        while(f1<=r1)
        {
            q2[r2++]=q1[f1++];
        }
        f1=r1=0;
        q1[r1]=x;
        while(f2 {
            q1[++r1]=q2[f2++];
        }
        f2=-1;
        r2=-1;
    }
}
void pop()
{
    if(f1==-1||f1>r1)
    {
        printf("Stack underflow\n");
    }
    else
    {
        printf("Popped: %d\n",q1[f1++]);
        if(f1>r1)
        {
            f1=-1;
            r1=-1;
        }
    }
}
void display()
{
    if(f1==-1||f1>r1)
```

```

{
printf("Stack is empty\n");
}
else
{
for(int i=f1;i<=r1;i++)
{
printf("%d",q1[i]);
}
printf("\n");
}
}
int main()
{
int ch,x;
while (1)
{
printf("1.Push\n2.Pop\n3.Display\n4.Exit\nEnter your choice: ");
scanf("%d",&ch);
switch (ch)
{
case 1:
printf("Enter element to push: ");
scanf("%d",&x);
push(x);
break;
case 2:
pop();
break;
case 3:
display();
break;
case 4:
return 0;
default:
printf("Invalid choice\n");
}
}
return 0;
}

```


Program 67: done

```
//done
//count the number of nodes in a linked list
#include
#include
struct node
{
int d;
struct node *n;
};
int main()
{
struct node *first=NULL;
struct node *second=NULL;
struct node *third=NULL;
first=(struct node *)malloc(sizeof(struct node));
second=(struct node *)malloc(sizeof(struct node));
third=(struct node *)malloc(sizeof(struct node));
first->d=1;
first->n=second;
second->d=2;
second->n=third;
third->d=3;
third->n=NULL;
struct node *ptr=first;
int c=0;
while(ptr!=NULL)
{
c++;
ptr=ptr->n;
}
printf("Number of nodes in the linked list: %d\n",c);
return 0;
}
```

Program 68: done

```
//done
#include
struct date
{
    int day;
    int month;
    int year;
};
struct employee
{
    int id;
    char name[20];
    float salary;
    struct date da;
} em;
int main()
{
    printf("Enter employee id: ");
    scanf("%d",&em.id);
    printf("Enter employee name: ");
    scanf("%s",em.name);
    printf("Enter employee salary: ");
    scanf("%f",&em.salary);
    printf("Enter employee date of joining: ");
    scanf("%d %d %d",&em.da.day,&em.da.month,&em.da.year);
    printf("Employee id: %d\n",em.id);
    printf("Employee name: %s\n",em.name);
    printf("Employee salary: %f\n",em.salary);
    printf("Employee date of joining: %d %d %d\n",em.da.day,em.da.month,em.da.year);
    return 0;
}
```

Program 69: done

```
//done
// create a linked list with user defined number of nodes
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
int main()
{
    int n;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(int i=0;i {
        struct node *new=(struct node*)malloc(sizeof(struct node));
        printf("Enter the data: ");
        scanf("%d",&new->d);
        new->n=head;
        head=new;
    }
    struct node *temp=head;
    while(temp!=NULL)
    {
        printf("%d ",temp->d);
        temp = temp->n;
    }
    return 0;
}
```

Program 70: done

```
//done
// to check wether a list is a plaindrome or not(with least usage of space and time
complexity)
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void create(int data)
{
struct node *new=(struct node *)malloc(sizeof(struct node));
new->d=data;
new->n=head;
head=new;
}
void checkplaindrome()
{
struct node *slow=head;
struct node *fast=head;
while(fast!=NULL&&fast->n!=NULL)
{
slow=slow->n;
fast=fast->n->n;
}
struct node *mid=slow;
struct node *prev=NULL;
struct node *curr=mid;
struct node *next=NULL;
while(curr!=NULL)
{
next=curr->n;
curr->n=prev;
prev=curr;
curr=next;
}
mid=prev;
struct node *left=head;
struct node *right=mid;
while(right!=NULL)
{
if(left->d!=right->d)
{
printf("not plaindrome\n");
return;
}
left=left->n;
right=right->n;
}
```

```
}
printf("plaindrome\n");
return;
}
int main()
{
    int n,data;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(int i=0;i {
        printf("Enter the data for node %d: ",i+1);
        scanf("%d",&data);
        create(data);
    }
    checkplaindrome();
    return 0;
}
```

Program 71: done

```
//done
//Polynomial Representation using Linked List
#include
#include
struct node
{
    int coef;
    int exp;
    struct node *next;
};
struct node *head=NULL;
void insert(int coef,int exp)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->coef=coef;
    new->exp=exp;
    new->next=NULL;
    if(head==NULL)
        head=new;
    else
    {
        struct node *temp=head;
        while(temp->next!=NULL)
        {
            temp=temp->next;
        }
        temp->next=new;
    }
}
void display()
{
    struct node *temp=head;
    while(temp!=NULL)
    {
        if(temp->next!=NULL)
            {printf("%dx^%d+",temp->coef,temp->exp);}
        else
            {printf("%dx^%d",temp->coef,temp->exp);}
        temp=temp->next;
    }
    printf("\n");
}
int main()
{
    int n,i,coef,exp;
    printf("Enter the number of terms in the polynomial: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter the coefficient %d: ",i+1);
        scanf("%d",&coef);
```

```
printf("Enter the exponent %d: ", i+1);
scanf("%d", &exp);
insert(coef, exp);
}
printf("The polynomial is: ");
display();
return 0;
}
```

Program 72: done

```
//done
//reverse an array
#include
int main()
{
    int n;
    printf("Enter the size of the array: ");
    scanf("%d", &n);
    int arr[n],i;
    printf("Enter the elements of the array");
    for(i=0;i {
        scanf("%d",&arr[i]);
    }
    for(i=n-1;i>=0;i--)
    {
        printf("%d ",arr[i]);
    }
    return 0;
}
```


Program 73: deletion of a node in a doubly linked list by the value in the node

//deletion of a node in a doubly linked list by the value in the node

```
#include
#include
struct node
{
    int d;
    struct node *p;
    struct node *n;
};
struct node *head=NULL;
struct node *tail=NULL;
void insert(int data)
{
    struct node *new=(struct node*)malloc(sizeof(struct node));
    new->d=data;
    new->n=NULL;
    if(head==NULL)
    {
        head=new;
        tail=new;
        new->p=NULL;
    }
    else
    {
        tail->n=new;
        new->p=tail;
        tail=new;
    }
}
void delete(int data)
{
    struct node *temp=head;
    if(temp==NULL)
    {
        return;
    }
    if(temp->d==data)
    {
        head=temp->n;
        if(head!=NULL)
        {
            head->p=NULL;
        }
        free(temp);
        return;
    }
    while(temp->n!=NULL)
    {
        if(temp->n->d==data)
        {
```

```

struct node *del=temp->n;
temp->n=del->n;
if(del->n!=NULL)
{
del->n->p=temp;
}
else
{
tail=temp;
}
free(del);
return;
}
temp=temp->n;
}
}
void display()
{
if(head==NULL)
{
printf("List is empty\n");
}
else
{
struct node *temp=head;
while(temp!=NULL)
{
printf("%d->",temp->d);
temp=temp->n;
}
printf("NULL\n");
}
}
int main()
{
int n,v,i,data;
printf("Enter the number of nodes in the linked list: ");
scanf("%d",&n);
for(i=0;i {
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
insert(data);
}
display();
printf("Enter the value of the node to be deleted: ");
scanf("%d",&v);
delete(v);
display();
return 0;
}

```

Program 74: create an input restricted queue and perform enqueue dequeue and display operations

```
// create an input restricted queue and perform enqueue dequeue and display operations
#include
int q[100];
int f=-1;
int r=-1;
void enqueue(int x)
{
    if(r==99)
    {
        printf("Queue is full\n");
    }
    else
    {
        r=r+1;
        q[r]=x;
    }
}
void dequeue(int c)
{
    if(f==-1&&r==-1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        if(c==1)
        {
            printf("Deleted item from front of the queue is:%d\n",q[f]);
            f++;
        }
        else
        {
            printf("Deleted item from rear of the queue is:%d\n",q[r]);
            r--;
        }
    }
}
void display()
{
    int i;
    if(f==-1&&r==-1)
    {
        printf("Queue is empty\n");
    }
    else
    {
        for(i=f;i<=r;i++)
        {
```

```

printf("%d ",q[i]);
}
printf("\n");
}
}
int main()
{
int a,c,x;
while(1)
{
printf("Enter 1 for enqueue\nEnter 2 for dequeue\nEnter 3 for display\nEnter 4 to
exit\n");
printf("Enter your choice: ");
scanf("%d",&c);
switch(c)
{
case 1:
printf("Enter the element to enqueue: ");
scanf("%d",&x);
enqueue(x);
break;
case 2:
printf("Enter 1 to delete from front\nEnter 2 to delete from rear\n");
scanf("%d",&a);
dequeue(a);
break;
case 3:
display();
break;
case 4:
return 0;
default:
printf("Invalid choice\n");
}
}
}
}

```

Program 75: done

//done

//create a stack with an array and perform push pop and display on it

#include

int top=-1;

int stack[100];

void push(**int** x)

{

if(top<99)

{

top=top+1;

stack[top]=x;

}

else

{

printf("stack overflow\n");

}

}

void pop()

{

if(top>=0)

{

top=top-1;

}

else

{

printf("stack underflow\n");

}

}

void display()

{

int i;

for(i=top;i>=0;i--)

{

printf("%d\n",stack[i]);

}

}

int main()

{

int value;

printf("Enter the number of elements: ");

scanf("%d",&value);

for(**int** i=0;i {

printf("Enter element %d: ",i+1);

scanf("%d",&stack[i]);

}

top=value-1;

display();

pop();

display();

```
printf("Enter the push value: ");  
scanf("%d",&value);  
push(value);  
display();  
return 0;  
}
```

Program 76: Given the head of a linked list and the number k, Your task is to find the kth node from the end. If k is more than the number of nodes, then the output should be -1.

//Given the head of a linked list and the number k, Your task is to find the kth node from the end. If k is more than the number of nodes, then the output should be -1.

```
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int x)
{
    struct node *temp=(struct node*)malloc(sizeof(struct node));
    temp->d=x;
    temp->n=NULL;
    if(head==NULL)
    {
        head=temp;
    }
    else
    {
        struct node *p=head;
        while(p->n!=NULL)
        {
            p=p->n;
        }
        p->n=temp;
    }
}
void display()
{
    struct node *p=head;
    while(p!=NULL)
    {
        printf("%d->", p->d);
        p=p->n;
    }
    printf("NULL \n");
}
int kne(int k)
{
    struct node *p=head, *q=head;
    int count=0;
    while(p!=NULL)
    {
        p=p->n;
```

```

count++;
}
if(count {
return -1;
}
else
{
for(int i=0;i {
q=q->n;
}
return q->d;
}
}
int main()
{
int n,i;
printf("Enter the number of nodes: ");
scanf("%d",&n);
for(i=0;i {
int data;
printf("Enter the data for node %d: ",i+1);
scanf("%d",&data);
insert(data);
}
display();
int k;
printf("Enter the value of k: ");
scanf("%d",&k);
int result=kne(k);
if(result== -1)
{
printf("Node does not exist \n");
}
else
{
printf("The %dth node from the end is %d \n",k,result);
}
return 0;
}

```


Program 77: first make a loop out of the user defined linked list from the user defined point and then show it and then delete the loop

```
//first make a loop out of the user defined linked list from the user defined point and
then show it and then delete the loop
//Remove loop in linked list
#include
#include
struct node
{
int d;
struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
struct node *new=(struct node*)malloc(sizeof(struct node));
new->d=data;
new->n=NULL;
if(head==NULL)
{
head=new;
}
else
{
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=new;
}
}
void display()
{
struct node *temp=head;
int c=0;
while(temp!=NULL&& c<20)
{
printf("%d ",temp->d);
temp=temp->n;
c++;
}
if(temp!=NULL)
{
printf("-----list is cyclic\n");
}
printf("\n");
}
void createLoop(int pos)
{
if(pos<=0)
return;
```

```

struct node *h=head;
int c=1;
while(h!=NULL&& c; {
h=h->n;
c++;
}
if(h==NULL)
{
printf("Invalid position\n");
return;
}
struct node *temp=head;
while(temp->n!=NULL)
temp=temp->n;
temp->n=h;
printf("Loop created: Last node now points to node at position %d.\n", pos);
}
void rmloop()
{
struct node *slow=head;
struct node *fast=head;
int le=0;
while(fast!=NULL&&fast->n!=NULL)
{
slow=slow->n;
fast=fast->n->n;
if(slow==fast)
{
le=1;
break;
}
}
if(le!=1)
{
printf("No loop detected.\n");
return;
}
slow=head;
if(slow==fast)
{
while(fast->n!=slow)
fast=fast->n;
}
else
{
while(slow->n!=fast->n)
{
slow=slow->n;
fast=fast->n;
}
}
fast->n=NULL;

```

```
printf("The loop has been removed.\n");
}
int main()
{
    int n,i,data,pos;
    printf("Enter the number of nodes in the linked list: ");
    scanf("%d",&n);
    for(i=0;i {
        printf("Enter data for node %d: ",i+1);
        scanf("%d",&data);
        insert(data);
    }
    printf("Linked list: ");
    display();
    printf("Enter the position to create a loop: ");
    scanf("%d",&pos);
    createLoop(pos);
    printf("The looped linked list:\n");
    display();
    rmloop();
    printf("Linked list after loop removal: ");
    display();
    return 0;
}
```

Program 78: Given a singly linked list. The task is to remove duplicates (nodes with duplicate values) from the given list (if it exists). Note: Try not to use extra space. The nodes are arranged in a sorted way.

//Given a singly linked list. The task is to remove duplicates (nodes with duplicate values) from the given list (if it exists). Note: Try not to use extra space. The nodes are arranged in a sorted way.

```
#include
#include
struct node
{
    int d;
    struct node *n;
};
struct node *head=NULL;
void insert(int data)
{
    struct node *temp=(struct node*)malloc(sizeof(struct node));
    temp->d=data;
    temp->n=NULL;
    if(head==NULL)
    {
        head=temp;
    }
    {
        struct node *temp1=head;
        while(temp1->n!=NULL)
        {
            temp1=temp1->n;
        }
        temp1->n=temp;
    }
}
struct Node* removeDuplicates(struct Node* head)
{
    struct Node* current = head;
    struct Node* next = NULL;
    struct Node* prev = NULL;
    while (current != NULL)
    {
        next = current->n;
        while (next != NULL && next->d == current->d)
        {
            next = next->n;
        }
        if (next == current->n)
        {
            prev = current;
            current = next;
        }
    }
}
```

```
else
{
prev->n = next;
current = next;
if(prev!=NULL)
{
next = prev->n;
}
else
}
}
}
}
```

Program 79: create a program to find the factorial of a number using recursion and iteration give choice of method to user

//create a program to find the factorial of a number using recursion and iteration give choice of method to user

```
#include
#include
int fact(int n)
{
    if (n==0)
        return 1;
    else
        return n*fact(n-1);
}
int main()
{
    long double n,c;
    clock_t start,end;
    double t1,t2;
    printf("Enter a number to find its factorial: ");
    scanf("%lf",&n);
    printf("Enter 1 for recursion or 2 for iteration: ");
    scanf("%lf",&c);
    if(c==1)
    {
        start=clock();
        printf("Factorial of %lf is %d\n",n,fact(n));
        end=clock();
        t1=(double)(end-start);
        t2=t1/CLOCKS_PER_SEC;
        printf("Time taken: %f seconds\n",t2);
    }
    else if(c==2)
    {
        start=clock();
        {
            int i,r=1;
            for (i=1;i<=n;i++)
                r*=i;
            printf("Factorial of %lf is %d\n",n,r);
        }
        end=clock();
        t1=(double)(end-start);
        t2=t1/CLOCKS_PER_SEC;
        printf("Time taken: %f seconds\n",t2);
    }
    else
    {
        printf("Invalid choice\n");
    }
    return 0;
}
```

}